A Parameterized Output-Stationary Systolic Array for Fixed-Point Matrix Multiplication: Phase 1 Prototype

Gokul VKS, Milind Bansal, Isha Tomar, Yashi Agrawal and Mohammad Affan Anas
Newton School of Technology, Rishihood University
Code and results: https://github.com/vks-g/systolic-matmul-accelerator

***Abstract*—Matrix multiplication dominates the run time of neural-network inference and many signal-processing workloads, and on general-purpose processors its cost is set by data movement rather than arithmetic. This paper presents the Phase 1 prototype of a matrix-multiplication accelerator built on an output-stationary systolic array. An $N \times N$ grid of processing elements, 4×4 by default, multiplies signed 8-bit fixed-point operands and accumulates each element of the result in place in a 32-bit register. Operands enter through two skew buffers, so each row of A and each column of B reaches the array one cycle later than the previous one. A five-state controller sequences the computation. A host loads A and B through write ports, pulses `start` and reads C after `done`. The design is synthesizable, parameterized Verilog-2005 with a written timing contract: a product takes $K+2N-2$ compute cycles, 10 for a 4×4 array, and 12 cycles from `start` to `done`. That is 16 multiply-accumulates per cycle at peak, and 6.4 times fewer compute cycles than a single sequential multiply-accumulate unit. Every module is checked by a self-checking testbench that runs in continuous integration. The complete accelerator matches a Python golden model on all 106 directed and random test cases, and its cycle counter reads the predicted 12 cycles in every run.**

***Index Terms*—systolic array, matrix multiplication, hardware accelerator, processing element, output-stationary dataflow, fixed-point arithmetic, Verilog**

I. Introduction

Matrix multiplication is the core operation of fully connected and convolutional neural-network layers, of transformers, and of many filters and transforms in signal processing. Multiplying two $N \times N$ matrices takes N^3 multiply-accumulate (MAC) operations, and each operand is used N times. A general-purpose processor fetches those operands again and again through its memory hierarchy, and the energy of that data movement exceeds the energy of the arithmetic by orders of magnitude [1], [2]. Dedicated hardware that keeps operands close to many parallel MAC units is therefore the standard way to accelerate this workload.

A systolic array is a regular grid of identical processing elements (PEs), each connected only to its neighbours, through which data moves in step with a single clock [3], [4]. Each operand is read from memory once and then reused as it passes from PE to PE, so the array performs many operations per memory access. Its short, local wires make it straightforward to scale and to lay out. The same structure is at the heart of production accelerators such as Google's Tensor Processing Unit [5] and of open-source generators such as Gemmini [6].

This project designs, verifies and evaluates a systolic-array matrix-multiplication accelerator in two phases. Phase 1, reported here, delivers a working register-transfer-level (RTL) prototype:

- a parameterized $N \times N$ output-stationary systolic array, 4×4 by default, for signed 8-bit fixed-point operands with 32-bit accumulators;
- input skew buffers, an output buffer and a controller, which together give the array a simple host interface: write A and B, pulse `start`, read C;
- a written specification that fixes every port, the input skew, and a compute latency of $K+2N-2$ cycles before any RTL was written, so the five team members could build and test their modules in parallel;
- a verification flow in which every module has a self-checking testbench, a Python golden model provides the expected results, and continuous integration lints and simulates every pull request.

Phase 2 will synthesize the design, measure its speed-up against a sequential baseline in RTL, and extend it to matrices larger than the array.

Section II reviews systolic arrays, their dataflows and published accelerators. Section III describes the architecture, the PE and multiply-accumulate unit, the dataflow and the controller. Section IV presents the verification methodology and Section V the status of the prototype. Section VI lists the contributions of each team member.

II. Literature Review

A. Systolic Arrays

Kung and Leiserson introduced systolic arrays as grids of simple, identical cells that pass data only to their nearest neighbours, with every cell working in step with one clock [3]. Kung later argued that the main benefit is reuse: an operand fetched from memory once is used by many cells as it moves through the array, so the array can do much more computation per memory access [4]. Matrix multiplication fits this model well, because each element of A and B is used N times in an $N \times N$ product. Short, local wires and a regular layout also make the design easy to scale and to lay out in silicon.

The reuse argument matters more today than in 1982. At 45 nm, an off-chip DRAM access costs more than three orders

TABLE I
SYSTOLIC AND SPATIAL ACCELERATORS COMPARED WITH THIS WORK

Design	Dataflow	PEs	Operands
TPU [5]	WS	256×256	8-bit int
ShiDianNao [9]	OS	8×8	16-bit fixed
Eyeriss [10]	RS	12×14	16-bit fixed
Gemmini [6]	OS or WS	configurable	configurable
This work	OS	4×4 (any N)	8-bit int

TABLE II
DESIGN PARAMETERS

Parameter	Meaning	Default
N	array size; A, B and C are $N \times N$	4
K	inner dimension (equal to N in Phase 1)	4
DATA_W	operand width, signed	8
ACC_W	accumulator and result width, signed	32
FRAC_W	fractional bits of the Q-format reading	0
T	compute cycles, $K + 2N - 2$	10

of magnitude more energy than an 8-bit integer multiply [1]. An accelerator saves energy by keeping data on chip and close to the arithmetic, and a systolic array does this by design.

B. Dataflows

Chen *et al.* classify accelerator dataflows by which value stays in place inside each processing element (PE) [2], [7]. In a *weight-stationary* (WS) dataflow, each PE holds one weight while inputs flow past and partial sums move out. In an *output-stationary* (OS) dataflow, each PE holds one output and accumulates it in place while both operands flow through. *Row-stationary* (RS) and *no-local-reuse* dataflows trade between these. SCALE-Sim, a cycle-level simulator for systolic accelerators, adds *input-stationary* (IS) and shows that the best dataflow and array shape depend on the layer dimensions [8]. No dataflow is best for every workload.

OS suits a small matrix-multiplication array for two reasons. First, partial sums never leave the PE, so the array needs no adder chain between PEs and no partial-sum buffer. Second, the full result C is already in place when computation ends, so it can be read out directly. The cost is pipeline fill and drain. An $N \times N$ OS array computing an inner dimension of K needs $K+2N-2$ cycles, of which only K keep any single PE busy. This is why utilization is low when K is small, 40 % for $N = K = 4$, and approaches full use as K grows.

C. Accelerators Built on These Dataflows

Table I compares several published designs with this work. Google's first Tensor Processing Unit (TPU) uses a 256×256 WS systolic array of 8-bit multiply-accumulate units, with 32-bit accumulators, for datacenter neural-network inference [5]. ShiDianNao uses an OS 8×8 PE array placed next to an image sensor, and keeps each output neuron in its PE while inputs shift between neighbours [9]. Eyeriss introduced the RS dataflow on a 168-PE chip and reduces data movement for convolutions [7], [10]. Gemmini is an open-source generator of systolic-array accelerators whose array can run either OS or WS, chosen at run time, so the two can be compared on the same hardware [6].

D. Fixed-Point Arithmetic

Integer arithmetic is much cheaper than floating point in both energy and area [1]. Jacob *et al.* show that neural networks can run inference with 8-bit integer weights and activations and 32-bit integer accumulators, with little loss of accuracy [11]. The TPU uses the same 8-bit operand and 32-bit accumulator widths [5]. This work follows that choice. With 8-bit signed operands and $K = 4$, the largest possible sum needs 18 bits, so a 32-bit accumulator cannot overflow.

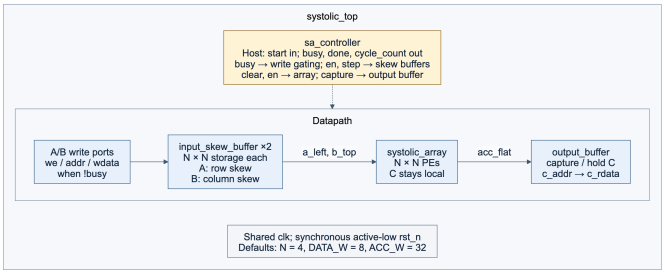


Fig. 1. System architecture. Solid arrows carry data and dashed arrows carry controller signals. Host writes reach the input skew buffers only while `busy` is low. Each buffer stores one matrix and selects its skewed lanes from `step`. The controller clears the array, enables it for T steps and then captures the accumulators into the output buffer, which serves the read port.

E. Position of This Work

The designs above are built for throughput on large workloads. This prototype is a small, fully parameterized OS array whose timing contract (input skew, a compute latency of $K+2N-2$ cycles and a start/busy/done handshake) is stated precisely and checked against a Python golden model that also simulates the array cycle by cycle. The literature suggests three directions for Phase 2: tiling to support $K \neq N$ and larger matrices, which improves utilization; comparing OS with WS on the same array, as Gemmini allows; and buffering inputs and outputs so that loading and readout overlap with computation.

III. PROPOSED SOLUTION

A. Architecture Overview

The accelerator computes $C = AB$, where A and B are $N \times N$ matrices of signed DATA_W-bit operands and

$$C[i][j] = \sum_{k=0}^{K-1} A[i][k]\, B[k][j]. \tag{1}$$

In Phase 1 the inner dimension K equals N. Table II lists the parameters. Every module takes them as Verilog parameters, so the array size and the data widths change in one place.

The design has five blocks behind one host interface, shown in Fig. 1:

- **Two input skew buffers.** Each stores one operand matrix and feeds one edge of the array. The A buffer drives row

i of A into the left edge i cycles late. The B buffer drives column j of B into the top edge j cycles late.

- **The systolic array.** An $N \times N$ grid of PEs. PE (i, j) multiplies the operands that reach it, adds the product to its accumulator, which holds $C[i][j]$, and passes A to the right and B down one cycle later. The PE is described in Section III-B and the skew in Section III-C.
- **The output buffer.** Copies all N^2 accumulators in one cycle when a computation ends, and serves them to the host through a read port. The previous result therefore stays readable while the next product is computed.
- **The controller.** A five-state machine that clears the array, runs the T compute steps, and signals the capture and the end of the run (Section III-D).

The host sees a small matrix data interface. It writes A and B one element at a time through two write ports with row-major addresses, pulses `start`, waits for `done`, and reads C one element at a time through the read port. While `busy` is high, writes are ignored, so the operands of a running product cannot change. Between runs A and B keep their contents, so a host can replace one matrix and start again. A `cycle_count` output reports how many cycles the last run took, which the testbenches use to check the latency.

A run proceeds as follows. On `start` the controller spends one cycle clearing every accumulator and forwarding register. It then runs $T = K + 2N - 2$ compute steps, during which the skewed operands cross the array and every PE adds its K products. One more cycle copies the accumulators into the output buffer, and `done` rises. For $N = K = 4$, `busy` is high for $K + 2N = 12$ cycles.

The array performs $N^2 = 16$ MACs per cycle at its peak and computes the $N^2K = 64$ MACs of a 4×4 product in 10 compute cycles, against 64 cycles for a single sequential MAC unit: a speed-up of 6.4 on compute and 5.3 over all 12 busy cycles. Each PE is busy for K of the T cycles, so utilization is $K/T = 40\,\%$ at this size, because the wavefront needs $2N-2$ cycles to fill and drain the array. Utilization approaches $100\,\%$ as K grows. With one-element ports, loading A and B takes 16 cycles and reading C another 16. A complete product including I/O therefore takes about 44 cycles, so at this size the narrow ports, not the array, limit throughput.

The arithmetic is exact integer arithmetic: the 32-bit accumulators cannot overflow at these sizes (Section III-B), so the datapath needs no rounding or saturation. Operands can also be read as Q-format fixed-point numbers with FRAC_W fractional bits, and each element of C then has $2\,$FRAC_W fractional bits. This changes only how the numbers are read, not the hardware.

B. Processing Element and MAC Unit

Each processing element (PE) owns one output element $C[i][j]$. It holds a multiply-accumulate unit (MAC) and two forwarding registers. The MAC multiplies two signed 8-bit operands into a 16-bit product, sign-extends that product to a 32-bit accumulator, and adds it to the running sum in every enabled cycle. The product is formed at its natural width before the sign extension, so the sign bit is never read as zero. Clear has priority over enable, and reset behaves like clear.

The PE also registers its operands. The left operand a is copied to the right neighbour and the top operand b to the neighbour below, one enabled cycle later. Each operand therefore moves one PE per cycle, which is what lets the skewed inputs meet at PE (i, j) in the cycle that the term $A[i][k]B[k][j]$ needs. Fig. 2 shows both behaviours in the PE testbench: the forwarded operands trail the inputs by one cycle, and four products accumulate to the expected dot product of 70.

The accumulator width follows from the largest possible sum. For $K = N = 4$ the largest magnitude is $K \cdot 2^{2\cdot\mathrm{DATA_W}-2} = 4 \cdot 2^{14} = 65{,}536$, reached when every operand is -128. That needs 18 bits, so a 32-bit accumulator cannot overflow for any K below 131,072. The datapath therefore does not saturate. Rounding and saturating back to 8 bits are left for Phase 2.

C. Dataflow and Input Skewing

The array is output-stationary: PE (i, j) keeps $C[i][j]$ in its accumulator for the whole run, and only the operands move. Row i of A enters the left edge at PE $(i, 0)$ and moves one PE to the right per cycle. Column j of B enters the top edge at PE $(0, j)$ and moves one PE down per cycle. Results never move during the computation, so the output buffer can copy all N^2 of them in one cycle when it ends.

The two operands of a product must reach a PE in the same cycle. Number the compute cycles $t = 0, 1, \ldots, T-1$. In step t the skew buffers put $A[i][t-i]$ on lane i of the left edge and $B[t-j][j]$ on lane j of the top edge, and zero when the index falls outside $0 \le k < K$; Table III lists them for $N = 4$. Each PE adds one cycle of delay in each direction, so in step t PE (i, j) receives

$$a = A[i][t-i-j], \qquad b = B[t-i-j][j]. \tag{2}$$

Both carry the same index $k = t - i - j$. PE (i, j) therefore adds the term $A[i][k]\,B[k][j]$ of (1) in step $t = i + j + k$, and it receives zeros, which add nothing, in every other step.

The compute length follows from the last term. PE (i, j) is busy from step $i + j$ to step $i + j + K - 1$, so the last PE, $(N-1, N-1)$, adds its last term in step $K + 2N - 3$, and

$$T = K + 2N - 2, \tag{3}$$

which is 10 for $N = K = 4$. The PEs that are busy in one step lie on anti-diagonals: in step 3, PE $(0, 0)$ adds its term $k = 3$ while PE $(3, 0)$ adds its term $k = 0$. The edges go quiet after step $K + N - 2 = 6$, but the last operands are still crossing the array until step 9.

Each input skew buffer stores one $N \times N$ matrix, written row-major by the host before `start`. During compute it selects its lanes combinationally from the controller's `step`: lane l carries $M[l][t-l]$ for A, whose lanes are rows, and $M[t-l][l]$ for B, whose lanes are columns. One module serves both edges through a parameter. The buffer compares $t \ge l$ before it subtracts, so the index never wraps around. Selecting

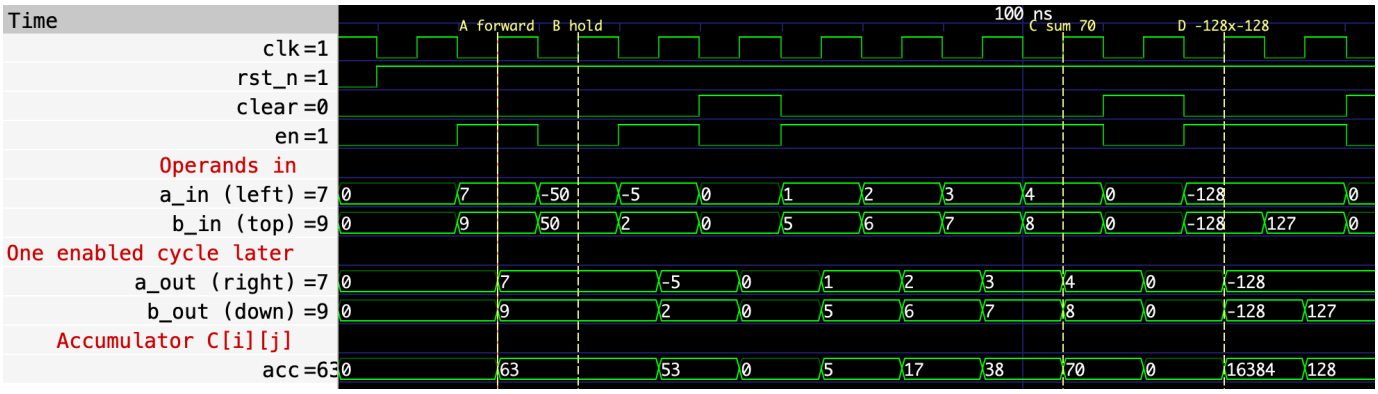


Fig. 2. One PE in simulation (`tb_pe`, GTKWave). Each operand leaves on `a_out` and `b_out` one enabled cycle after it arrives (A) and holds while `en` is low (B). The accumulator sums $1 \cdot 5 + 2 \cdot 6 + 3 \cdot 7 + 4 \cdot 8 = 70$ over $K = 4$ steps (C), and -128×-128 gives 16,384 (D).

TABLE III
EDGE INPUTS IN EACH COMPUTE STEP FOR $N = K = 4$ ($\cdot$ IS ZERO)

t	left edge, lane i				top edge, lane j			
	0	1	2	3	0	1	2	3
0	A_{00}	$\cdot$	$\cdot$	$\cdot$	B_{00}	$\cdot$	$\cdot$	$\cdot$
1	A_{01}	A_{10}	$\cdot$	$\cdot$	B_{10}	B_{01}	$\cdot$	$\cdot$
2	A_{02}	A_{11}	A_{20}	$\cdot$	B_{20}	B_{11}	B_{02}	$\cdot$
3	A_{03}	A_{12}	A_{21}	A_{30}	B_{30}	B_{21}	B_{12}	B_{03}
4	$\cdot$	A_{13}	A_{22}	A_{31}	$\cdot$	B_{31}	B_{22}	B_{13}
5	$\cdot$	$\cdot$	A_{23}	A_{32}	$\cdot$	$\cdot$	B_{32}	B_{23}
6	$\cdot$	$\cdot$	$\cdot$	A_{33}	$\cdot$	$\cdot$	$\cdot$	B_{33}
7–9	$\cdot$	$\cdot$	$\cdot$	$\cdot$	$\cdot$	$\cdot$	$\cdot$	$\cdot$

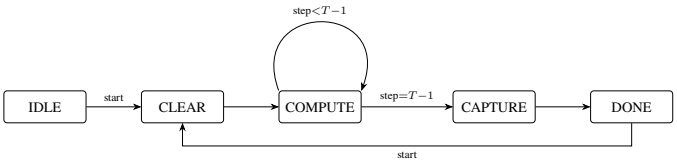


Fig. 3. Controller state-transition diagram. `busy` is high in `CLEAR`, `COMPUTE` and `CAPTURE`.

from stored operands rather than shifting them through delay registers means the host may write the matrices in any order, and both matrices stay in place for the next run.

D. Controller

The controller sequences one matrix multiplication through five states, shown in Fig. 3. Its outputs are decoded from the state register, so they change only on clock edges. In `CLEAR` the accumulators and forwarding registers are zeroed. `COMPUTE` then runs for $T = K + 2N - 2$ cycles, which is 10 for $N = 4$: the last PE, $(N-1, N-1)$, adds its last product in step $T - 1$. `CAPTURE` copies every accumulator into the output buffer, and `DONE` holds the result while `done` is high. A `start` pulse is accepted only in `IDLE` and `DONE` and is ignored while the array is busy. A full run keeps the controller busy for $K + 2N = 12$ cycles, and `cycle_count` reports that number when the run ends.

IV. Verification Methodology

Verification checks both the values of $C = AB$ and the timing and handshake contract in `docs/spec.md`. Tests progress from individual modules to the array and the complete accelerator, so a failure can be traced to an arithmetic, forwarding, skew or control boundary.

A. Tool Flow

The synthesizable Verilog-2005 RTL first passes static checks with Verilator [12]. The `make lint` target discovers all RTL files and runs `--lint-only -Wall`, selecting each module as a top and using the Verilog-2005 language setting. Warnings fail the target; intentional unused signals have local waivers with explanations.

The `make test` target generates golden vectors, compiles each `tb/tb_*.v` with Icarus Verilog using `-g2005 -Wall`, and runs the resulting executable with `vvp` [13]. Testbenches dump VCD waveforms into `build/`. GTKWave provides manual inspection of the operand wavefront, accumulator updates and controller transitions [14]; saved layouts are in `sim/waves/`. The `make waves` target reruns the top-level test and opens its VCD in the selected viewer. Waveform inspection supplements the automated comparisons. Synthesis, resource counts and maximum-frequency measurement are Phase 2 work and are not part of this simulation flow.

B. Hierarchical Testbenches

The MAC test checks signed products, including -128×-128 and -128×127, an accumulation chain, reset, clear priority and holding with enable low. The PE test adds the one-enabled-cycle forwarding delay and checks that both forwarding registers hold and clear with the accumulator. Input-buffer tests compare every A and B lane at each step with the spec's skew table; output-buffer tests check parallel capture, holding the previous result and readout at every address.

Controller tests check the state sequence, busy-state rejection of `start`, restart from `DONE`, mid-run reset and the exact compute and busy lengths for $N = 3$ and $N = 4$. Array tests drive skewed operands at the edges and check every PE's partial sum after each compute edge, as well as

final results, stalls and restart. They exercise both array sizes, catching row/column indexing and fill/drain errors that a single final-result comparison could miss.

The end-to-end test loads operands through the host ports and compares every output element with golden vectors. It also checks ignored writes while busy, back-to-back runs, replacing only one operand matrix, preservation of the previous C until capture, and reset during a run. For each golden case it counts busy cycles and checks `cycle_count`: both must equal $K + 2N$, or 12 for $N = K = 4$. The CSV in `results/phase1/cycle_counts.csv` records these measurements; its compute-cycle field is derived by subtracting the one CLEAR and one CAPTURE cycle. The controller tests check the compute interval directly against $T = K + 2N - 2$, which is 10 cycles at the default size.

C. Golden Results and Directed Cases

The Python reference in `model/golden_model.py` computes signed matrix products with NumPy integer arithmetic and checks operand and result ranges. A separate cycle-level model starts with zero forwarding registers and accumulators, applies the specified input skew, and advances operands one PE per cycle. The vector generator refuses to write its files unless this model agrees with the mathematical product after exactly T steps. Expected results therefore come from the reference or hand-worked values, rather than recorded RTL output.

The default vector set contains six directed cases—identity, zero, all ones, maximum negative operands, mixed signed extremes and a known product with $A = 1, \ldots, 16$ in row-major order and $B = A^{\mathsf{T}}$—plus 100 random cases spanning the signed 8-bit range. The golden-vector seed is 1; array tests use seeds 1 and 2 for $N = 4$ and $N = 3$. Seeds are printed in simulation logs, and the vector manifest maps cases to file offsets. Separate Python self-tests check the reference, cycle model, hexadecimal encoding and generator against hand-worked results and the spec.

D. Failure Reporting and Reproduction

Each self-checking testbench prints the following summary on success:

```
PASS tb_<module> <passed>/<total>
```

A successful test terminates with `$finish`. A mismatch reports a failure and calls `$fatal(1, ...)`, producing a nonzero simulator exit. Make also rejects a `FAIL` summary or a missing, correctly formatted `PASS` summary. Make retains compilation, lint and simulation logs in the generated build directory. Repeated Make invocations rerun simulations even when compiled executables already exist. Generated vectors and VCDs remain uncommitted; selected logs, cycle counts and waveform captures in `results/phase1/` provide report evidence.

E. Continuous Integration and Scope

The GitHub Actions simulation workflow runs on every pull request and on pushes to `dev`. On Ubuntu 24.04 it selects Python 3.12, installs Icarus Verilog, Verilator and NumPy, then executes the same `make lint test` command used locally. Failed jobs upload available `build/*.log` files as diagnostic artifacts. The Python model self-tests, GUI viewer and report build are separate local checks; they are not steps in this workflow.

TABLE IV
TESTBENCH RESULTS, `MAKE LINT TEST` ON `DEV`

Testbench	Module under test	Checks passed
`tb_mac`	MAC unit	20/20
`tb_pe`	processing element	15/15
`tb_sa_controller`	controller	56/56
`tb_input_skew_buffer`	input skew buffer	496/496
`tb_output_buffer`	output buffer	96/96
`tb_systolic_array`	4×4 and 3×3 array	7,471/7,471
`tb_systolic_top`	complete accelerator	2,195/2,195
Total		10,349/10,349

All seven available RTL testbenches pass the current regression, including all 106 golden cases end to end. These checks establish the observed functional and cycle-level behavior for the exercised cases; they are not an exhaustive proof or a measurement of physical timing. Phase 2 synthesis will assess implementation resources and timing.

V. Prototype Status

Every Phase 1 module is complete and integrated: the MAC unit, the PE, the $N \times N$ array, the input skew buffers, the output buffer, the controller and the top level. All seven RTL files pass Verilator lint with every warning enabled, and every module has a self-checking testbench. Table IV lists the checks each testbench makes in the current regression, `make lint test` on `dev`: 10,349 checks, none failed. The same command runs in continuous integration on every pull request. The logs of the buffer, array and end-to-end tests are kept in `results/phase1/logs/`.

The end-to-end test drives the accelerator only through its host ports. It loads each of the 106 golden cases, runs it, and compares every element of C with the golden model, so every result of every case matches. It also checks the handshake rules: writes during a run are ignored, back-to-back runs work, one matrix can be replaced on its own, C stays readable until the next capture, and a reset during a run returns the accelerator to idle.

A. Measured Latency

For every case the testbench counts the cycles in which `busy` is high and reads `cycle_count` when `done` rises. Both are 12 in all 106 runs, recorded in `results/phase1/cycle_counts.csv`: one `CLEAR` cycle, $T = 10$ compute cycles and one `CAPTURE` cycle, exactly as the timing contract predicts. Fig. 4 shows the computation inside the array on the known product: the operands enter along the skewed edges and the results fill in diagonal by diagonal, with PE $(3, 3)$ finishing in the last compute step.

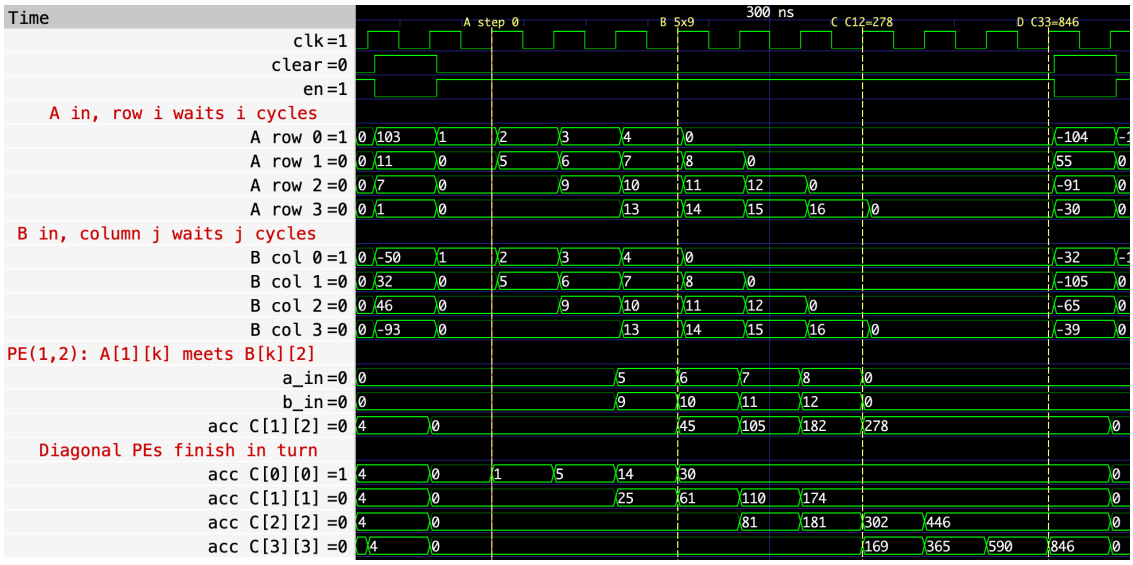


Fig. 4. The 4×4 array on the known product ($A = 1, \ldots, 16$ in row-major order, $B = A^{\mathsf{T}}$), from `tb_systolic_array`. Row i of A and column j of B enter i and j cycles late, so $A[1][k]$ meets $B[k][2]$ at PE $(1, 2)$ from marker B on and gives $C[1][2] = 278$ (C). PE $(3, 3)$ finishes $C[3][3] = 846$ in the last step, $T - 1 = 9$ (D). Values during `CLEAR` are random and ignored.

TABLE V
MEASURED ARRAY LATENCY AGAINST ONE SEQUENTIAL MAC ($N = K = 4$)

Metric	Sequential MAC	Systolic array
MACs per product	64	64
Compute cycles	64	10
Cycles from `start` to `done`	–	12
MACs per compute cycle, average	1	6.4
MACs per cycle, peak	1	16
Speed-up, compute cycles	1	6.4
Speed-up, `start` to `done`	1	5.3

B. Comparison with a Sequential MAC

A single sequential MAC unit needs one cycle per multiply-accumulate, so an $N \times N$ product with $K = N$ takes $N^3 = 64$ cycles. Table V compares it with the measured array. The array performs the same 64 MACs in 10 compute cycles, 6.4 on average per cycle against a peak of 16, so it needs 6.4 times fewer compute cycles, and 5.3 times fewer counting all 12 busy cycles. The gap grows with the array: the sequential unit needs N^3 cycles and the array $3N - 2$, so for $N = 8$ the compute ratio would be $512/22 \approx 23$. These baseline figures are analytical; a sequential RTL baseline, synthesis and clock-frequency results are Phase 2 work. Loading the operands and reading the result through the one-element ports adds 32 cycles per product, so at this size the ports, not the array, limit end-to-end throughput.

VI. Individual Contributions

Every module, testbench, model script and report section has one owner, assigned through a GitHub issue. Each member committed their own work through pull requests into a shared integration branch, and every pull request was reviewed before merging. Table VI lists who did what.

TABLE VI
INDIVIDUAL CONTRIBUTIONS

Member	Contribution
Gokul VKS (lead)	Architecture specification and interface contract; systolic array grid and its testbench; top-level integration; repository, issues and reviews; abstract, introduction, architecture overview and report assembly
Milind Bansal	Multiply-accumulate unit and processing element; controller state machine; their testbenches; waveform captures; report sections on the PE and the controller
Isha Tomar	Input skew buffers and output buffer with their testbenches; end-to-end accelerator testbench and cycle counts; video script and edit; report sections on the dataflow and the prototype status
Yashi Agrawal	Python golden model and test-vector generator; literature review; bibliography
Mohammad Affan Anas	Toolchain setup guide, Makefile and continuous-integration workflow; architecture diagrams; README; verification methodology section; clean-clone release check

References

[1] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE International Solid-State Circuits Conference (ISSCC) Digest of Technical Papers*, 2014, pp. 10–14.

[2] V. Sze, Y.-H. Chen, T.-J. Yang, and J. S. Emer, "Efficient processing of deep neural networks: A tutorial and survey," *Proceedings of the IEEE*, vol. 105, no. 12, pp. 2295–2329, 2017.

[3] H. T. Kung and C. E. Leiserson, "Systolic arrays (for VLSI)," in *Sparse Matrix Proceedings 1978*, I. S. Duff and G. W. Stewart, Eds. Philadelphia, PA, USA: SIAM, 1979, pp. 256–282.

[4] H. T. Kung, "Why systolic architectures?" *Computer*, vol. 15, no. 1, pp. 37–46, 1982.

[5] N. P. Jouppi, C. Young, N. Patil, D. Patterson *et al.*, "In-datacenter performance analysis of a tensor processing unit," in *Proc. 44th Annual*

International Symposium on Computer Architecture (ISCA), 2017, pp. 1–12.

[6] H. Genc, S. Kim, A. Amid, A. Haj-Ali, V. Iyer, P. Prakash, J. Zhao, D. Grubb, H. Liew, H. Mao *et al.*, "Gemmini: Enabling systematic deep-learning architecture evaluation via full-stack integration," in *Proc. 58th ACM/IEEE Design Automation Conference (DAC)*, 2021.

[7] Y.-H. Chen, J. Emer, and V. Sze, "Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks," in *Proc. ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*, 2016, pp. 367–379.

[8] A. Samajdar, J. M. Joseph, Y. Zhu, P. Whatmough, M. Mattina, and T. Krishna, "A systematic methodology for characterizing scalability of DNN accelerators using SCALE-Sim," in *Proc. IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, 2020, pp. 58–68.

[9] Z. Du, R. Fasthuber, T. Chen, P. Ienne, L. Li, T. Luo, X. Feng, Y. Chen, and O. Temam, "ShiDianNao: Shifting vision processing closer to the sensor," in *Proc. 42nd Annual International Symposium on Computer Architecture (ISCA)*, 2015, pp. 92–104.

[10] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, "Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks," *IEEE Journal of Solid-State Circuits*, vol. 52, no. 1, pp. 127–138, 2017.

[11] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2018, pp. 2704–2713.

[12] Verilator Project, "Verilator user's guide: Executable arguments," https://verilator.org/guide/latest/exe_verilator.html, accessed: Nov. 14, 2026.

[13] Icarus Verilog Project, "Simulation using Icarus Verilog," https://steveicarus.github.io/iverilog/usage/simulation.html, accessed: Nov. 14, 2026.

[14] GTKWave Authors, "GTKWave documentation: Introduction," https://gtkwave.github.io/gtkwave/, accessed: Nov. 14, 2026.